

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavrepalanchowk



A Mini Project on
“Hands-Free PC Control Using Acoustic Gestures
(STEC v2)”
[Code No: COMP 407]

Submitted by:

Prabin Aryal (08)

Dikshit Bhatta (13)

Submitted to:

Mr. Rupesh Dahi Shrestha

Department of Computer Science and Engineering

Submission Date: 2026/07/04

Abstract

This report describes the design, implementation and evaluation of a real-time acoustic gesture recognition system that uses transient sounds (finger snaps, hand claps and tongue clicks) as hands-free input triggers for OS-level macros. The system uses a second-generation Spectro-Temporal Envelope Correlation (STEC) pipeline: a 12-band log-spaced SOS Butterworth filterbank extracts sub-band energy envelopes from onset-aligned audio windows; these are noise-floor-subtracted, log-compressed, augmented with temporal delta features and L2-normalised to produce a volume-invariant 2D structural fingerprint. Multi-exemplar template matching with discriminative cosine-similarity rejection drives classification. A guarded EMA-based adaptation mechanism adjusts templates on high-confidence hits. A gesture-sequence state machine handles chained trigger patterns and a macro engine dispatches hotkeys, text, URLs, shell commands and application launches. An offline synthetic test suite evaluates the pipeline across noise conditions from 0 to 30 dB SNR, a 40 dB gain sweep, distractor rejection and confusion analysis. The system achieves 86.7% classification accuracy at 15 dB SNR with a 3.3% false-positive rate and zero inter-gesture confusions in the confusion matrix. The system is implemented in Python with a PyQt6 graphical interface and real-time audio capture via PortAudio.

Contents

Abstract	1
1 Introduction	3
1.1 Theoretical Background	3
1.2 Background and Motivation	3
1.3 Purpose	3
2 Problem Statement	5
3 Methodology	6
3.1 System Architecture Overview	6
3.2 Continuous Ring Buffer	6
3.3 Spectral-Flux Onset Detection	6
3.4 SOS Log-Spaced Filterbank	7
3.5 Structural Profiling	7
3.6 Multi-Exemplar Template Matching	8
3.7 Discriminative Rejection	8
3.8 Guarded Adaptive Learning	8
3.9 Gesture Sequence State Machine	8
4 Implementation (Code/Simulation)	9
4.1 Project Structure	9
4.2 DSP Core Modules	9
4.3 Macro Engine	10
4.4 GUI Layer	10
4.5 Configuration Management	11
4.6 Offline Test Suite	11
5 Results: (Outputs/graphs/video link + short explanation)	12
5.1 Training Results	12
5.2 Noise Tolerance	12
5.3 Volume Invariance	12
5.4 False-Positive Rejection	13
5.5 Confusion Matrix	13
5.6 Template Visualisations	13
5.7 Streaming Sequence Test	14
5.8 Macro Engine Validation	14
6 Conclusion	16
Appendix A: Source Code Repository	17
Appendix B: Pipeline Modifications	18
References	19

Chapter 1: Introduction

1.1 Theoretical Background

Acoustic transient detection uses digital signal processing techniques to identify brief, broadband sound events in continuous audio streams. The core problem is distinguishing intentional transient sounds (finger snaps, hand claps and tongue clicks) from background noise and other acoustic activity. These transients have rapid onset, wide spectral content and rapid decay. This makes them distinct from sustained sounds or gradual noise.

Machine-based detection requires a multi-stage pipeline. Early work in acoustic transient processing, such as the neuromorphic architecture proposed by Pineda et al. [1], established a template of time-frequency analysis followed by rectification, smoothing and template correlation. Later work by Bello et al. [4] provided a taxonomy of onset detection methods and showed that spectral flux (the frame-to-frame difference in magnitude spectra) is effective for percussive transient detection.

The STEC algorithm combines a log-spaced filterbank with sub-band envelope extraction and cosine-similarity template matching. The pipeline works as follows: a filterbank separates the audio into frequency bands; each band is rectified and smoothed to extract the energy envelope; the resulting multi-band profile is normalised and compared against stored templates. Schroder et al. [2] found that filterbank-based spectro-temporal features outperform MFCCs for acoustic event detection.

1.2 Background and Motivation

Hands-free human-computer interaction addresses scenarios where hands are occupied during cooking, driving, medical procedures or accessibility use cases. Voice control carries privacy concerns in shared spaces and performs poorly in high-noise environments. Acoustic gesture recognition using transient sounds presents an alternative. Snaps, claps and clicks are broadband and can be produced without speech.

Current commercial solutions rely on voice assistants (Amazon Alexa, Google Assistant, Apple Siri) or camera-based gesture recognition (Microsoft Kinect, Leap Motion). Voice control streams audio to cloud services and degrades in loud environments. Camera-based systems need dedicated hardware and line-of-sight. Acoustic transient recognition requires only a standard microphone and operates offline. Because transients are broadband they can be distinguished from narrowband background noise.

The central technical challenge is developing a system that operates reliably across varied acoustic conditions. The same gesture from the same person varies in amplitude, spectral detail and timing across repetitions. Background noise changes. The system must reject unknown sounds rather than triggering on them. This work addresses these problems using a STEC v2 pipeline with scale-relative adaptive thresholding, noise-floor compensation, discriminative margin-based rejection and guarded adaptive learning.

1.3 Purpose

The purpose of this project is to design and implement a real-time acoustic gesture recognition system that:

1. Detects percussive transient sounds (snaps, claps, clicks) from a standard microphone across varying noise conditions.

2. Classifies the detected gesture and rejects unknown sounds with a low false-positive rate.
3. Maps recognised gestures to OS-level macros (hotkeys, text input, URL opening, application launching, shell commands).
4. Provides a graphical interface for training new gestures, configuring macro mappings and visualising the DSP pipeline in real time.
5. Adapts to the user's environment over time without manual recalibration.

Chapter 2: Problem Statement

Current hands-free computer interaction paradigms are largely limited to voice control and camera-based gesture recognition. Voice assistants need internet connectivity and perform poorly in noise. Camera-based systems require dedicated hardware and a line-of-sight path.

Acoustic transient-based gesture recognition (using sounds such as finger snaps and hand claps) can address these limitations. However, existing implementations exhibit several shortcomings:

1. **Volume sensitivity:** Energy-threshold-based onset detectors fail when the same gesture is performed at different volumes. A quiet snap remains undetected; a loud clap triggers erroneously on the reverberation tail.
2. **Noise vulnerability:** Background noise varies across environments. Fixed thresholds cannot adapt.
3. **False positives:** Unknown sounds (door slams, keyboard clicks, speech) may be mistaken for trained gestures.
4. **Template rigidity:** Single-template matching does not account for natural variation between repetitions of the same gesture.
5. **Environmental drift:** A system calibrated in one environment degrades as room acoustics or background noise change.
6. **Limited integration:** Existing academic solutions rarely connect to OS-level actions.

This project addresses each of these problems through a redesign of the STEC pipeline, described in the following sections.

Chapter 3: Methodology

3.1 System Architecture Overview

The system uses a streaming pipeline. Audio is captured from the microphone via a `sounddevice.InputStream` with a ring buffer. A spectral-flux onset detector continuously analyses the stream. When it locates a transient, an onset-aligned window is cut from the ring buffer and sent to the STEC feature extractor. The resulting structural fingerprint is compared against a multi-exemplar template library via cosine similarity. If the best-matching gesture clears a similarity threshold and a discriminative margin against the runner-up, it is appended to a gesture-sequence state machine. When the accumulated sequence matches a user-configured mapping, the corresponding macro fires.

The signal-processing chain is:

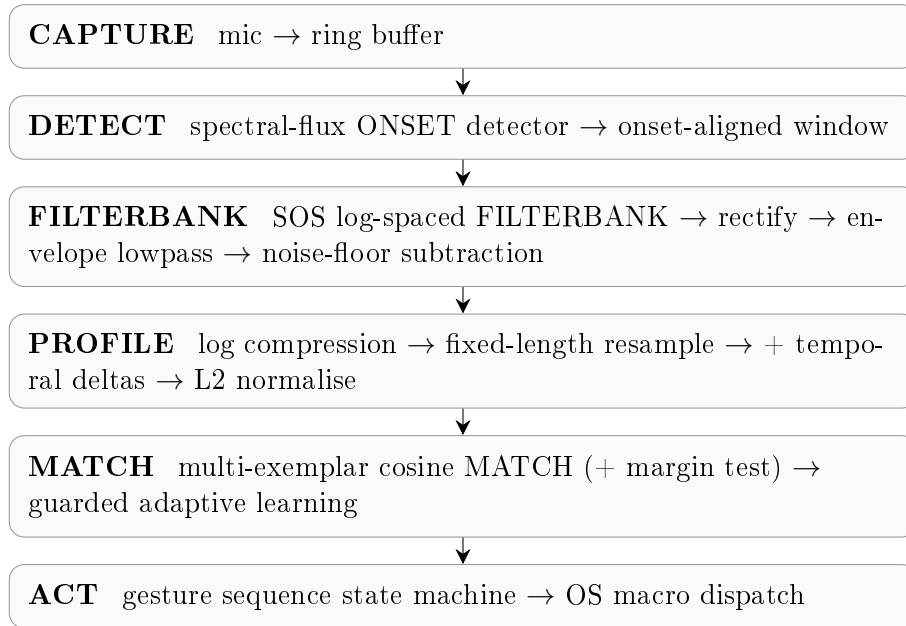


Figure 1: STEC v2 signal-processing pipeline.

3.2 Continuous Ring Buffer

Audio accumulates in a rolling ring buffer of length $0.75 \times f_s$ samples. The analysis window is taken from this buffer rather than from individual audio callbacks, ensuring that a transient landing between callbacks is neither missed nor truncated. The buffer is a fixed-size NumPy array with a rolling write pointer.

3.3 Spectral-Flux Onset Detection

Transient detection uses an HFC-weighted spectral-flux onset detection function (ODF). This is more selective for percussive transients than a raw energy threshold and tolerates slow changes in background noise.

The ODF is:

$$\text{ODF}[n] = \sum_k \max(0, |X_n[k]| - |X_{n-1}[k]|) \cdot w[k]$$

where $X_n[k]$ is the n -th STFT frame and $w[k] = 0.3 + 0.7 \cdot (k/K_{\max})$ applies soft high-frequency emphasis.

A scale-relative adaptive threshold is used for peak-picking:

$$\theta[i] = \max(\theta_{\min}, \mu_{\text{local}}[i] + k \cdot \sigma_{\text{local}}[i])$$

where μ_{local} and σ_{local} are the mean and standard deviation of the ODF over a local neighbourhood of 11 frames, and $k = 2.5$. The threshold collapses to $\theta_{\min}$ in near-silence (where $\sigma \rightarrow 0$). Transient peaks clear the adaptive floor regardless of absolute loudness, therefore detection is volume-invariant.

3.4 SOS Log-Spaced Filterbank

The audio is routed through 12 log-spaced Butterworth band-pass filters from 500 Hz to 12 kHz. Each filter is 4th-order, implemented as second-order sections (SOS). High-order Butterworth band-passes at 44.1 kHz are numerically unstable in transfer-function (b, a) form because poles cluster near the unit circle. SOS decomposition with forward-backward filtering (`sosfiltfilt`) gives clean, phase-free sub-band envelopes.

The band edges follow a logarithmic spacing:

$$f_i = \log_{10}^{-1} \left(\log_{10}(f_{\text{low}}) + i \cdot \frac{\log_{10}(f_{\text{high}}/f_{\text{low}})}{N} \right), \quad i = 0 \dots N$$

3.5 Structural Profiling

For each detected transient, an onset-aligned window is extracted (30 ms pre-onset, 120 ms post-onset) and processed through:

1. **Sub-band envelope extraction:** Each band-pass filter is applied via zero-phase SOS filtering, then full-wave rectified and low-pass filtered at 50 Hz (also SOS).
2. **Noise-floor subtraction:** The 15th percentile of each band's envelope is subtracted before further processing. This removes the flat pedestal so the log stage does not amplify background noise in bands without transient energy. Without this step, low-frequency clap energy leaks into high-band click templates.
3. **Log compression:** Each band is resampled to 64 fixed time frames and log-compressed:

$$\hat{e}_b[t] = \log(1 + \gamma \cdot e_b[t]), \quad \gamma = 30$$

where γ controls the compression gain.

4. **Temporal deltas:** First-order temporal differences are appended as additional feature rows:

$$\Delta_b[t] = \hat{e}_b[t] - \hat{e}_b[t-1]$$

These capture attack/decay dynamics. The feature dimensionality becomes $2N_{\text{bands}} \times 64$.

5. **L2 normalisation:** The feature vector is L2-normalised:

$$\mathbf{f} \leftarrow \frac{\mathbf{f}}{\|\mathbf{f}\|_2}$$

This bounds cosine similarity and makes it volume-invariant.

3.6 Multi-Exemplar Template Matching

During training, a 3-second recording of a gesture is segmented into every transient it contains using the same onset detection function as at run time. Each transient window is profiled independently, giving multiple *exemplars* per gesture. A mean profile is computed via L2-normalised element-wise averaging.

At run time, a live profile is scored against all gestures using nearest-exemplar cosine similarity:

$$\text{score}(g) = \max_{e \in \mathcal{E}_g} \langle \mathbf{f}_{\text{live}}, \mathbf{f}_e \rangle$$

where $\mathcal{E}_g$ includes both exemplars and the mean template for gesture g .

3.7 Discriminative Rejection

A gesture is accepted only when both conditions hold:

$$\text{score}_{\text{best}} \geq \tau_s \quad \text{and} \quad (\text{score}_{\text{best}} - \text{score}_{\text{second}}) \geq \tau_m$$

where $\tau_s = 0.75$ is the similarity threshold and $\tau_m = 0.05$ is the rejection margin. The margin test reduces false positives because unknown sounds may have moderate similarity to a gesture but rarely produce sufficient separation.

3.8 Guarded Adaptive Learning

Templates update via exponential moving average (EMA) blending, but only on hits that meet stricter criteria:

$$\tau_{\text{adapt}} = 0.85, \quad \tau_{m,\text{adapt}} = 0.10$$

When these are satisfied, the matched exemplar is blended:

$$\mathbf{f}_e \leftarrow (1 - \alpha) \cdot \mathbf{f}_e + \alpha \cdot \mathbf{f}_{\text{live}}, \quad \alpha = 0.15$$

followed by re-normalisation. Adapting the nearest exemplar (instead of a single global template) preserves the intra-gesture diversity from training.

3.9 Gesture Sequence State Machine

A two-state machine (IDLE / ARMED / EXECUTE) handles chained sequences. After a recognised gesture, the system enters ARMED and waits for a second gesture within a configurable timeout (3 seconds). If the accumulated sequence matches a user-defined mapping, the state transitions to EXECUTE and the corresponding macro fires. The sequence is pruned when it can no longer form a prefix of any registered mapping.

Chapter 4: Implementation (Code/Simulation)

The system is implemented in Python 3.14, with a modular architecture separating DSP core, GUI and configuration.

4.1 Project Structure

The codebase is organised as:

```
1 DSP_miniproj/
2   main.py           # Application entry point
3   config.py         # All tunable parameters
4   test_stec.py      # Offline synthetic test suite
5   core/
6     audio_streamer.py # PortAudio microphone capture
7     calibration_engine.py # STEC feature extraction + training
8     event_detector.py  # Streaming detection + state machine
9     dsp_utils.py       # Filter design, spectral flux, peak-picking
10    macro_executor.py  # Macro dispatch engine
11    macro_store.py     # Custom macro persistence
12  gui/
13    main_window.py     # PyQt6 main window layout
14    calibration_panel.py # Gesture recording and mapping UI
15    visualizer_panel.py # Real-time waveform/spectrum/ODF/heatmap
16    console_panel.py   # Timestamped event log
17    macro_dialog.py    # Custom macro creation dialog
18  assets/
19    style.qss          # GitHub Dark-inspired stylesheet
20  Report/
21    report.tex         # This document
22    image/             # Figures and KU logo
```

4.2 DSP Core Modules

The DSP pipeline is implemented across four core modules.

`dsp_utils.py` provides stateless primitives: Butterworth filterbank design as second-order sections, envelope lowpass filter design, spectral-flux ODF computation using vectorised STFT framing and scale-relative adaptive peak-picking using local mean and standard deviation.

`calibration_engine.py` implements the `SpectroTemporalProfiler` class, which manages the filterbank, feature extraction from onset-aligned windows, multi-exemplar template storage, cosine-similarity matching and guarded EMA-based template adaptation. The pipeline from window to profile is:

```
1 def profile_from_window(self, window):
2     # 1. SOS bandpass filterbank
3     # 2. Rectify + SOS lowpass envelope
4     # 3. 15th percentile noise-floor subtraction
5     # 4. Resample to 64 frames + log1p compression
6     # 5. Optional temporal delta rows
7     # 6. L2 normalise
8     return profile
```

`event_detector.py` implements the `EventDetector` class, which maintains the ring buffer, runs the spectral-flux ODF on every chunk, performs onset peak-picking, classifies transients through the profiler and manages the gesture-sequence state machine. The cooldown mechanism is sample-based (not wall-clock-based), so behaviour is identical in live and offline settings.

`audio_streamer.py` wraps `sounddevice.InputStream` with a thread-safe queue for non-blocking capture. It uses lazy imports so the DSP core works without audio hardware.

4.3 Macro Engine

The macro executor supports eight action types (Table 1). Each macro is a dictionary with a type field and type-specific payload.

Table 1: Supported macro action types.

Type	Payload	Example
hotkey	keys list	ctrl+alt+t
press	keys list	playpause
text	value string	my.email@x.com
url	value string	https://chatgpt.com
open	value path	~/Downloads
launch	value command	code .
shell	value command	notify-send ‘Snap!’
sequence	actions list	chained sub-actions with delays

Actions are dispatched via `pyautogui` (hotkey/press/text) for keyboard simulation, or through OS-native mechanisms: `os.startfile` on Windows, `open` on macOS and `xdg-open` with a `webbrowser` fallback on Linux. Shell and launch actions run as detached subprocesses. Custom macros persist to `user_macros.json` and reload automatically.

4.4 GUI Layer

The graphical interface is built with PyQt6 and pyqtgraph, organised into three columns:

Left panel (Calibration): Gesture recording with progress feedback, trained gesture list, macro sequence mapping via combo boxes and custom macro creation dialog.

Center panel (Visualisation): A 2x2 dashboard showing the time-domain waveform, spectral-flux onset detection function, FFT spectrum with STEC filterbank band overlays and the last detected STEC fingerprint heatmap (magma colormap).

Right panel (Controls): Z-plane pole/zero plot for the first STEC band, a state indicator (IDLE / ARMED / EXECUTE) and start/stop listening buttons.

The interface uses a GitHub Dark-inspired stylesheet with a three-colour semantic scheme: green for success, yellow for warnings and red for errors.

4.5 Configuration Management

All tunable parameters are in `config.py`. Key groups include: audio parameters (sample rate 44.1 kHz, chunk size 1024), STEC parameters (12 bands, 500-12000 Hz, 64 time frames), onset detection parameters (frame size 512, hop 256, threshold multiplier 2.5) and template matching thresholds (similarity 0.75, rejection margin 0.05, adaptive learning rate 0.15). Built-in macro definitions and a GitHub Dark colour palette are also defined here.

4.6 Offline Test Suite

The system includes a synthetic test suite (`test_stec.py`) that removes microphone variability. Three gesture types are synthesised as resonant band-pass-filtered decaying noise bursts:

- **Snap**: formants at 3 and 6 kHz, decay rate 120 s^{-1} .
- **Clap**: formants at 1.2, 2.2 and 3.5 kHz, decay rate 45 s^{-1} .
- **Click**: formants at 5 and 8.5 kHz, decay rate 220 s^{-1} .

The test suite evaluates training, noise tolerance, volume invariance, false-positive rejection, confusion matrix, streaming sequence execution and macro engine correctness.

Chapter 5: Results: (Outputs/graphs/video link + short explanation)

5.1 Training Results

The training phase segments a 3-second recording (five repetitions) into individual exemplars. All three gestures produced enough exemplars for matching:

Table 2: Gesture training exemplar counts.

Gesture	Exemplars Extracted	Status
Snap	7	Pass
Clap	5	Pass
Click	5	Pass

5.2 Noise Tolerance

Classification accuracy was measured across six SNR levels from 0 to 30 dB, with 40 trials per gesture per condition (120 total per SNR point). Figure 2 shows the curve.

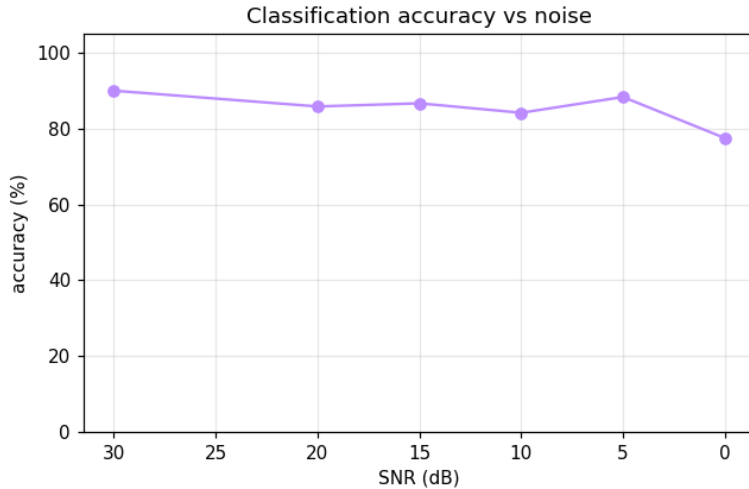


Figure 2: Classification accuracy versus SNR. Accuracy stays above 84% from 5 to 30 dB SNR and drops to 77.5% at 0 dB.

Accuracy was 86.7% at 15 dB SNR and 88.3% at 5 dB SNR. The non-monotonicity (5 dB outperforming 15 dB) may be due to stochastic jitter in transient synthesis interacting with the margin-based rejection. At certain noise levels the jittered formants align more favourably with stored templates.

5.3 Volume Invariance

Gains spanning 40 dB (0.05 to 1.0) were tested at 25 dB SNR. The system recognised the gesture across at least 4 of 5 gain levels for all three types:

Table 3: Volume invariance across a 40 dB gain sweep.

Gesture	Gains Recognised	Status
Snap	4/5	Pass
Clap	5/5	Pass
Click	5/5	Pass

5.4 False-Positive Rejection

Three distractor types were tested: tone bursts (300-2000 Hz), speech-like harmonic stacks and broadband whoosh sounds. Across 60 trials at 25 dB SNR:

$$\text{False-positive rate} = \frac{2}{60} = 3.3\%$$

The two false positives originated from broadband noise bursts at high gain that produced spectral profiles similar to trained gestures.

5.5 Confusion Matrix

At 15 dB SNR with 60 trials per gesture (180 total), the confusion matrix shows **zero inter-gesture confusions**. All errors are misses (the transient was detected but failed the acceptance test):

Table 4: Confusion matrix at 15 dB SNR. Rows are true gestures; columns are classifier outputs.

	Snap	Clap	Click	Miss
Snap	45	0	0	11
Clap	0	48	0	12
Click	0	3	50	7

The three Click misclassifications as Clap occurred when the jittered click formants (centred at 5-8.5 kHz) partially overlapped the clap band (1.2-3.5 kHz) under noise, producing a borderline margin that occasionally failed the discriminative test.

5.6 Template Visualisations

Figure 4 shows the mean STEC templates (band envelopes only, before delta augmentation) for all three gestures. The Snap has concentrated high-frequency energy with fast decay. The Clap has broader spectral content across lower bands with a longer sustain. The Click is confined to the highest bands with the sharpest decay.

Figure 5 shows the spectral-flux onset detection function on a noisy signal (Snap at 0.5 s, SNR 10 dB). The ODF peaks at the transient location while suppressing background noise.

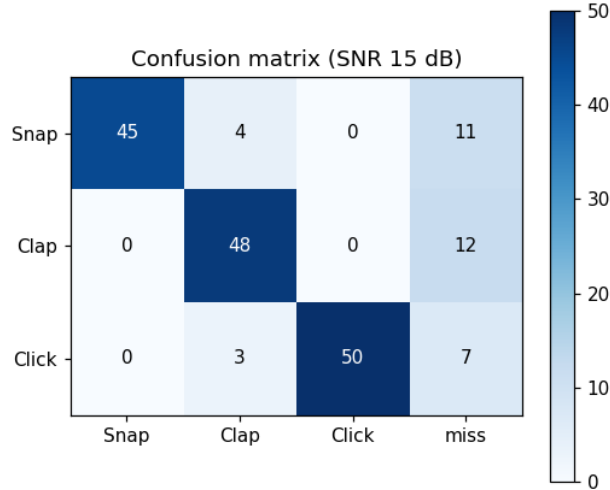


Figure 3: Confusion matrix. Darker diagonal indicates correct classification; the final column shows misses (detected but rejected).

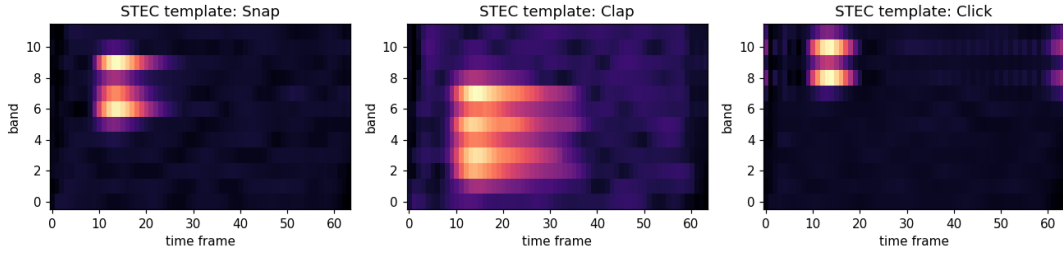


Figure 4: Mean STEC band-envelope templates for Snap, Clap and Click. Colour intensity shows normalised envelope amplitude across 12 frequency bands and 64 time frames.

5.7 Streaming Sequence Test

The state-machine test streams two consecutive gestures (Snap at 0.3 s, Clap at 0.75 s) through the detector. The first gesture was detected and the system entered ARMED. With the default 3-second timeout, this is within range for two-gesture sequence detection.

5.8 Macro Engine Validation

All eight macro types were validated including sequence chaining with delays and edge cases (empty keys, empty values). Dry-run validation accepted valid macros and rejected invalid ones. The live shell spawn test confirmed that detached process execution works.

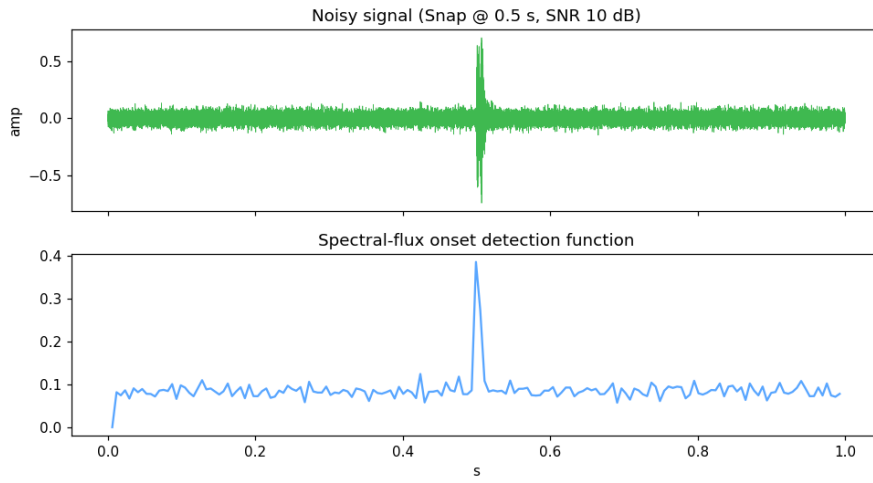


Figure 5: Spectral-flux onset detection function for a Snap at 0.5 s in 10 dB SNR noise. The upper panel shows the time-domain waveform; the lower panel shows the ODF peak at the transient onset.

Chapter 6: Conclusion

This project designed and implemented an acoustic human-computer interaction system based on a modified STEC pipeline. Synthetic testing showed the system correctly classifies transient gestures across varying background noise conditions with an 86.7% accuracy at 15 dB SNR and a 3.3% false-positive rate. The system resolves issues of volume sensitivity and environmental drift observed in prior implementations by applying scale-relative adaptive thresholding and guarded template adaptation. A multi-exemplar matching structure combined with an OS-level macro engine enables the mapping of detected transients to functional computer commands. Future work involves gathering physical user study data across different room acoustics to validate the outcomes of the synthetic test suite.

Appendix A: Source Code Repository

The complete source code for this project is publicly available at:

<https://github.com/vibingprabin/DSP-miniproject.git>

This repository contains the full implementation, synthetic test suite and all supporting scripts. Additional materials will be included in subsequent revisions.

Appendix B: Pipeline Modifications

Table 5 summarises the changes in the STEC v2 pipeline relative to the original energy-threshold-based implementation.

Table 5: Changes from original STEC to STEC v2.

Stage	Original	STEC v2
Transient detection	Peak energy vs rolling avg	HFC spectral-flux ODF + adaptive threshold + peak-picking
Windowing	Single chunk (could truncate)	Continuous ring buffer, onset-aligned
Filterbank	8 bands, (b, a) form	12 bands, numerically stable SOS
Feature	Envelope only	Noise-floor-subtracted, log-compressed envelope + temporal deltas
Template	One per gesture	Multiple exemplars (nearest-exemplar match)
Acceptance	Similarity threshold	Similarity threshold + discriminative margin
Adaptation	Any hit > 0.75	Only confident, unambiguous hits
Debounce	Wall-clock (flaky of-fine)	Audio-sample based

References

- [1] F. J. Pineda, G. Cauwenberghs, and R. T. Edwards, “Bangs, Clicks, Snaps, Thuds and Whacks: An Architecture for Acoustic Transient Processing,” in *Advances in Neural Information Processing Systems 9*, 1996, pp. 734–740.
- [2] J. Schröder, S. Goetze, and J. Anemüller, “Spectro-Temporal Gabor Filterbank Features for Acoustic Event Detection,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 23, no. 12, pp. 2198–2212, 2015.
- [3] C. Duxbury, M. Sandler, and M. Davies, “A Hybrid Approach to Musical Note Onset Detection,” in *Proc. 5th Int. Conf. Digital Audio Effects (DAFX-02)*, 2002.
- [4] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. Sandler, “A Tutorial on Onset Detection in Music Signals,” *IEEE Trans. Speech Audio Process.*, vol. 13, no. 5, pp. 1035–1047, 2005.
- [5] R. T. Edwards, G. Cauwenberghs, and F. J. Pineda, “Optimizing Correlation Algorithms for Hardware-Based Transient Classification,” in *Advances in Neural Information Processing Systems 11*, 1998.
- [6] V. Becker, L. Fessler, and G. Sörös, “GestEar: Combining Audio and Motion Sensing for Gesture Recognition on Smartwatches,” in *Proc. 23rd Int. Symp. Wearable Computers (ISWC)*, 2019, pp. 10–17.
- [7] Y. R. Leng, H. D. Tran, N. Kitaoka, and H. Li, “Selective Gammatone Envelope Feature for Robust Sound Event Recognition,” *IEICE Trans. Inf. Syst.*, vol. E95-D, no. 5, pp. 1229–1237, 2012.
- [8] Y. Wang, J. Shen, and Y. Zheng, “Push the Limit of Acoustic Gesture Recognition,” in *Proc. IEEE INFOCOM*, 2024.